Discrete Mathematics

Jaehaerys Wang

2026 年 3 月 11 日

目录

1 Chapter 1: Proofs **3**

1.1 什么是证明？ (What is a Proof?) 3

1.2 良序原理 (The Well Ordering Principle) 4

1.3 逻辑公式 (Logical Formulas) 4

1.4 数学数据类型 (Mathematical Data Types) 5

1.5 归纳法 (Induction) 6

1.6 递归数据类型 (Recursive Data Types) 7

1.7 无限集合 (Infinite Sets) 7

2 Chapter 2: Structures **8**

1 Chapter 1: Proofs

证明

本书的第一部分“证明”(Proofs)旨在为计算机科学提供坚实的数学和逻辑基础。该部分通过七个章节，系统地介绍了如何运用数学模型和方法来分析问题、建立确定性，并确保软件或算法的正确性。

从最基本的逻辑推演和证明技巧起步，逐步深入到良序原理和多种归纳法(尤其是与算法分析息息相关的状态机和结构归纳法)，最后上升到探讨无穷集合与计算边界的理论极限。这一部分为后续关于图论、图结构、计数理论和概率论等计算机科学内容的学习打下了严密的数学基石。

1. 什么是证明？ (What is a Proof?)
2. 良序原理 (The Well Ordering Principle)
3. 逻辑公式 (Logical Formulas)
4. 数学数据类型 (Mathematical Data Types)
5. 归纳法 (Induction)
6. 递归数据类型 (Recursive Data Types)
7. 无限集合 (Infinite Sets)

1.1 什么是证明？ (What is a Proof?)

基础概念: 介绍了**命题(Proposition)**，即非真即假的陈述，和**谓词(Predicate)**，即其真值取决于变量的命题。

公理化方法与推导: 证明是从一组**公理(Axioms)**出发，通过逻辑推导得出的结论链条。

常用证明模式: 详细讲解了几种基础的证明方法，包括:

- **蕴涵证明(Proving an Implication)**:证明“如果P，则Q”及其逆否命题。
- **当且仅当证明(If and Only If)**:证明双向蕴涵关系。
- **分类讨论(Proof by Cases)**:将复杂问题拆分为不同的情况分别证明。
- **反证法(Proof by Contradiction)**:假设命题为假并推导出逻辑矛盾。

经典的“伪证明”(Bogus Proofs)与逻辑陷阱: 强调培养**识别错误证明**的能力。

- **图形伪证明**:隐藏假设或误用图形来误导视觉判断。
- **计算伪证明**:忽视多义性或不当简化计算步骤。
- **归纳法伪证明**:骨牌断裂或错误的归纳假设。

1.2 良序原理 (The Well Ordering Principle)

核心定义: 任何非空的非负整数集合($\mathbb{N}$)都必然包含一个最小元素。虽然听起来极其直观，但它是离散数学中极其强大的证明工具。

应用: 提供了一种标准模板，通过假设存在反例集合，并由良序原理得出“最小反例”，随后推导出矛盾来证明原命题。例如，用于证明算术基本定理(整数可以分解为素数乘积)。

Well Ordering Principle Proofs: To prove $\forall n \in \mathbb{N}, P(n)$ using WOP:

1. Assume for contradiction that the set $S = \{n \in \mathbb{N} : \neg P(n)\}$ is non-empty.
2. By WOP, S has a least element, say m.
3. Show that $P(k)$ holds for all $k < m$ (this is often done by using the definition of m as the smallest counterexample).
4. Use the fact that $P(k)$ holds for all $k < m$ to derive a contradiction, thus proving that S must be empty and therefore $\forall n \in \mathbb{N}, P(n)$ holds.

1.3 逻辑公式 (Logical Formulas)

命题逻辑: 探讨了如何使用 AND($\wedge$)、OR($\vee$)、NOT($\neg$)、XOR($\oplus$)、IMPLIES($\rightarrow$)、IFF($\leftrightarrow$) 等逻辑连接词来组合命题，并通过真值表(Truth Table)来判断公式的等价性和有效性。

DeMorgan 定律: 介绍了 DeMorgan 定律，揭示了 NOT 与 AND/OR 之间的关系，即 $\neg(P \wedge Q) \equiv \neg P \vee \neg Q$ 和 $\neg(P \vee Q) \equiv \neg P \wedge \neg Q$。

可满足性问题(SAT Problem): 提及了判断命题公式是否可满足的计算复杂性问题。

- **可满足性(Satisfiability)**: 一个命题公式P是可满足的(有解的)，当且仅当在某些变量赋值下它为真，若P是可满足的则$\neg P$也是可满足的。
- **有效性(Validity)**: 一个命题公式P是有效的(永远为真)，当且仅当在所有变量赋值下它为真，若P是有效的则$\neg P$是不可满足的。
- **逻辑等价(Logical Equivalence)**: 两个命题公式P和Q是逻辑等价的，当且仅当在所有变量赋值下它们具有相同的真值。
- **蕴涵(Implication)**: 一个命题公式P蕴涵另一个命题公式Q，当且仅当在所有变量赋值下，如果P为真则Q也为真。
- **NP-完全(NP-Complete)**: NP指的是非确定性多项式时间复杂度类，可满足性问题是第一个被证明为 NP-完全的问题，这意味着它是所有 NP 问题中最难的一个，如果能找到一个多项式时间算法来解决 SAT 问题，那么所有 NP 问题都可以在多项式时间内解决。
- **$P = NP$ 问题**: 千禧年大奖难题之一，涉及是否所有可验证的解都可以在多项式时间内找到。解释了为什么快速检查命题公式是否可满足不仅是逻辑问题，更是理论计算机科学中最核心的未解之谜。

谓词公式: 引入了**全称量词**($\forall$，对于所有)和**存在量词**($\exists$，存在)，解释了多重量词嵌套时的顺序问题及其对应的逻辑含义。

关于逻辑公式的元定理(Meta-theorems):

- **完备性定理(Completeness Theorem)**: 任何在语义上有效的逻辑公式都可以通过形式证明系统来证明。换句话说，如果一个命题在所有模型中都为真，那么存在一个证明该命题的形式证明。即好消息是所有有效的逻辑公式都可以被证明。
- **不可判定性定理(Undecidability Theorem)**: 不存在一个算法能够判断所有逻辑公式是否有效。换句话说，无法构造一个程序来决定任意给定的命题是否在所有模型中为真。即坏消息是不存在一个算法能判断所有逻辑公式是否有效。
- **哥德尔不完全性定理(Gödel's Incompleteness Theorems)**: 在任何足够强大的形式系统中，存在无法被证明或反驳的命题。这表明了形式系统的局限性，即使是数学也无法完全自洽地证明所有真命题。

1.4 数学数据类型 (Mathematical Data Types)

基础结构: 严格定义了计算机科学中不可或缺的数学结构:

- **集合(Sets)**: 集合是元素的无序集合，强调了集合的基本操作(如并集$\cup$、交集$\cap$、差集$\setminus$、补集$\overline{A}$)和关系(如子集$\subseteq$、超集$\supseteq$)，以及特殊集合(如空集$\emptyset$)。
- **序列(Sequences)**: 序列是元素的有序列表，强调了序列的定义、长度以及常见操作(如连接、切片)。
- **函数(Functions)**: 函数是从一个集合到另一个集合的映射，强调了函数的定义域、值域、复合函数等概念，**映射(Mapping)**是函数的一种特殊类型。
- **二元关系(Binary Relations)**: 二元关系是集合元素之间的关系，强调了关系的性质(如自反性、对称性、传递性)以及特殊类型的关系(如等价关系、偏序关系)。

映射法则与基数:

- **基数(Cardinality)**: 集合的基数($|A|$)是集合中元素的数量。对于有限集合，基数就是元素的个数；对于无限集合，基数用特殊的符号表示，如自然数集$\mathbb{N}$的基数记为$\aleph_0$(阿列夫零)，实数集$\mathbb{R}$的基数大于$\aleph_0$，通常记作$c = 2^{\aleph_0}$。
- **有限集合(Finite Sets)**: 有限集合是指其基数为非负整数的集合，即包含有限数量元素的集合。
- **无限集合(Infinite Sets)**: 无限集合是指包含无限数量元素的集合。
- **可数集合(Countable Sets)**: 可数集合是指其基数与自然数集合$\mathbb{N}$相同的集合，即存在双射从该集合到$\mathbb{N}$的集合。
- **不可数集合(Uncountable Sets)**: 不可数集合是指其基数大于自然数集合$\mathbb{N}$的集合，即不存在双射从该集合到$\mathbb{N}$的集合。
- **满射(Surjection)**: 存在一个函数$f : A \to B$使得对于每个$b \in B$，至少存在一个$a \in A$使得$f(a) = b$。如果存在满射从集合A到集合B，则称集合A的基数大于或等于集合B的基数。

- **单射(Injection)**: 存在一个函数$f: A \to B$使得对于每个$a_1, a_2 \in A$，如果$f(a_1) = f(a_2)$，则$a_1 = a_2$。如果存在单射从集合A到集合B, 则称集合A的基数小于或等于集合B的基数。
- **双射(Bijection)**: 存在一个函数$f: A \to B$既是单射又是满射，即对于每个$b \in B$, 存在唯一的$a \in A$使得$f(a) = b$. 如果存在双射从集合A到集合B, 则称集合A和集合B具有相同的基数。
- **基数比较原则(Cardinality Comparison Principle)**: 如果存在单射从集合A到集合B, 则$|A| \leq |B|$；如果存在满射从集合A到集合B, 则$|A| \geq |B|$；如果存在双射从集合A到集合B, 则$|A| = |B|$。
- **施罗德-伯恩斯坦定理(Schröder-Bernstein Theorem)**: 如果存在单射从集合A到集合B，同时存在单射从集合B到集合A, 则存在双射从集合A到集合B, 因此集合A和集合B具有相同的基数。

1.5 归纳法 (Induction)

普通归纳法(Ordinary Induction): 归纳法是证明某属性$P(n)$对所有非负整数n成立的核心方法。

- **基础情况(Base Case)**: 验证$P(0)$或$P(1)$等初始值是否成立。
- **归纳步骤(Inductive Step)**: 假设$P(k)$对某个任意的非负整数k成立，证明$P(k+1)$也成立。则根据归纳法原理，$P(n)$对所有非负整数n成立。

强归纳法(Strong Induction): 强归纳法允许在归纳步骤中假设此前所有步骤均成立，从而简化证明。即在证明$P(k+1)$时，可以使用$P(0), P(1), \ldots, P(k)$的假设，而不仅仅是$P(k)$。

三大证明方法的等价性: **良序原理(WOP)、普通归纳法和强归纳法在逻辑上是完全等价的。** 任何一个用强归纳法写的证明，都可以通过添加全称量词机械地转换为普通归纳法证明；反之亦然。

状态机(State Machines): 状态机是一种step-by-step过程的抽象模型，适用于描述算法的执行过程。通过定义状态转移规则，可以使用归纳法来证明算法在每一步都满足某些不变性条件，从而确保算法的正确性和终止性。

- **Floyd不变性原理(Floyd's Invariant Principle)**: 在状态机的每一步中，如果某个属性(如数值)保持不变或满足特定条件，那么可以通过归纳法证明该属性在整个过程中始终成立。
- **衍生变量(Derived Variables)**: 定义一个严格递减的变量，如果该变量只能取非负整数值，那么根据良序原理，状态机必然会在有限步骤内终止。

1.6 递归数据类型 (Recursive Data Types)

递归定义(Recursive Definitions): 递归数据类型是通过自身定义的结构，常见于计算机科学中的字符串、树、表达式等。证明递归数据类型的性质通常使用结构归纳法。
结构归纳法(Structural Induction): 结构归纳法是归纳法的一种推广，适用于递归定义的数据结构。证明时只需涵盖基础元素，并证明构造操作能保持该属性不变即可。

- **基础情况(Base Case)**: 验证递归定义中的基本元素(如空字符串、单个节点)是否满足属性。
- **构造步骤(Constructive Step)**: 假设属性对某些元素成立，证明通过递归构造操作生成的新元素也满足该属性。

1.7 无限集合 (Infinite Sets)

对角线论证(Diagonal Argument): 康托尔对角线法，通过构造一个与任意给定列表中的元素都不同的元素，证明了实数集的不可数性。这个方法不仅揭示了无限集合的层次结构，还在计算机科学中被用来证明停机问题的不可判定性，以及罗素悖论等逻辑悖论的存在。
康托尔定理(Cantor's Theorem): 康托尔定理表明，对于任何集合A，其幂集$\mathcal{P}(A)$的基数严格大于A的基数，即$|\mathcal{P}(A)| > |A|$。这一结果揭示了无限集合的层次结构，说明了存在不同大小的无限集合，并且没有一个最大无限集合。这一发现对数学基础和集合论的发展产生了深远影响。
停机问题(Halting Problem): 通过对角线论证，证明了不存在一个算法能够判断任意程序是否会在有限时间内停止。这一结果表明了计算的根本限制，说明了某些问题是无法通过算法解决的，进而说明**完美的类型检查器或代码优化器**在在逻辑上是不存在的。
罗素悖论(Russell's Paradox): 罗素悖论揭示了在集合论中，如果允许任意集合的存在，就会导致矛盾。例如，考虑集合R，定义为所有不包含自身的集合的集合。问题是：R是否包含自身？如果R包含自身，那么根据定义它不应该包含自身；如果R不包含自身，那么根据定义它应该包含自身。这种矛盾表明了需要对集合的定义进行限制，最终促使数学家发展出更严格的集合论公理系统，如 ZFC 集合论，以避免类似的悖论。
ZFC 集合论: 通过一系列严格的公理来定义集合的存在和操作，避免了罗素悖论等逻辑矛盾，使得集合论成为一个坚实的数学基础。

- **Zermelo-Fraenkel 公理(ZF Axioms)**: 包括空集公理、配对公理、并集公理、幂集公理、替代公理等，定义了集合的基本性质和构造方法。
- **选择公理(Axiom of Choice)**: 断言对于任何非空集合的集合，都存在一个选择函数能够从每个集合中选择一个元素。选择公理在许多数学领域中具有重要应用，但也引发了一些争议，因为它允许构造一些非构造性的对象。

2 Chapter 2: Structures

TODO:结构章节内容待补充